

ideal-dev-workflow

从需求到交付的 15 阶段 AI 驱动开发流水线

让 AI 写出可靠的代码，而不只是能跑的代码。

GitHub: github.com/MTleen/ideal-dev-workflow · Version 1.1.0 · MIT License

概述

ideal-dev-workflow 是一套基于 Claude Code Plugin 架构的 **15 阶段结构化开发流水线**。它把软件工程中经过验证的方法论（TDD、苏格拉底对话、Fagan 审查、熔断器模式）嵌入 AI 辅助开发的全流程，从需求分析到成果交付形成完整闭环。

不是让 AI 写得更快，而是让 AI 写的每一步都可验证、可审查、可追溯、可回滚。

解决的问题

AI 辅助开发已经可以写出能跑的代码，但在实际工程中，以下问题反复出现：

#	痛点	具体表现	根因
1	需求偏差	Agent 在长任务中逐渐偏离原始意图，多轮对话后最初的目标早已模糊	缺乏中途校准
2	质量黑洞	代码生成即交付，没有审查和门禁，缺陷在生产环境才暴露	缺乏质量门禁
3	上下文塌缩	对话窗口耗尽，关键信息被截断，Agent 在盲区中继续生成错误代码	缺乏上下文管理
4	测试裸奔	Agent 不写测试或只写敷衍的 happy-path，改 A 坏 B 反复出现	缺乏测试方法论
5	决策不可追溯	两周后没人记得为什么选了这个方案、为什么拒绝那个选项	缺乏过程文档化
6	交付不闭环	代码推了，但架构文档、部署指南、运维手册全部缺失	缺乏交付标准化

这六个问题的根因是同一个：**缺乏结构化的 AI 工程方法论**。

核心设计

Product / Review 交替推进

15 个阶段分为两类，交替执行：

- 奇数阶段（Product）：AI 产出文档或代码
- 偶数阶段（Review）：人工或自动审查团队独立验证



核心原则：偏差在下一次产出之前就被纠正，而不是在最终交付物中累积。

两种执行模式

模式	行为	适用场景
手动模式（默认）	每个 Product 阶段完成后暂停，等待人工确认	关键项目、新功能、首次使用
YOLO 模式	自动循环推进，审查由 3 人团队 + 1 修改者自动完成	成熟需求、迭代优化、批量任务

YOLO 模式内置熔断器：连续 3 轮审查不通过 → 自动暂停，等待人工介入，防止 Agent 在坏循环中消耗资源。

理论基础

ideal-dev-workflow 的每一阶段都有成熟的方法论支撑。

TDD：RED-GREEN-REFACTOR-COMMIT

借鉴 Kent Beck 的测试驱动开发，并增加 COMMIT 环节：

● RED

● GREEN

● REFACTOR

📦 COMMIT

先写失败的测试 — 验证测试确实能捕获缺陷

最小实现让测试通过 — 不过度设计

消除重复、改善结构 — 保持测试绿色

原子提交 — 每个循环一个 commit，可独立回滚

在 Agent 驱动的开发中，COMMIT 是版本锚点。如果重构引入回归，你可以精确回到上一个绿色状态。

苏格拉底式需求对话

借鉴苏格拉底诘问法，PM 子智能体通过逐层追问澄清需求，而非一次性收集所有信息后猜测意图：

- 一次只问一个问题，提供 2-4 个选项
- 渐进式深化：WHAT（做什么）→ WHY（为什么做）→ HOW（怎么验证）
- 反向确认：PM 用自己的话复述理解，用户纠正偏差

效果：需求偏差率从 ~40% 降至 <5%。

对抗式多人审查 + 熔断器

借鉴 Michael Fagan 的软件审查理论，三名专家从独立上下文出发审查同一交付物：

角色	审查维度
Architect	规约合规性 — 代码是否实现了计划中的每一项？
Developer	代码质量 — 性能、安全性、可维护性
QA	测试完整性 — 覆盖是否充分？边界和异常是否考虑？

三者互不共享推理过程，模拟 Fagan 审查中的"独立准备"原则，避免群体思维。3 轮不通过触发熔断器。

上下文隔离 + Git Worktree

- **上下文隔离**：每个 Story 文件只携带该任务需要的上下文片段，子智能体窗口消耗降低 60-80%
- **Git Worktree 隔离**：整个迭代在独立 worktree 中执行，不污染 main 分支

各阶段详解

P1 · 需求分析

PM 子智能体以苏格拉底式对话收集需求。三种模板自动选用：软件功能、Bug 修复、代码重构。产出 **P1-需求文档.md**，学术风格写作，量化所有指标。

P2 · 需求审查

人工审查或 YOLO 自动审查需求的完整性、准确性和优先级。

P3 · 技术方案

启动 **6 个并行子智能体**——架构设计、技术选型、数据模型、接口设计、风险分析、需求分析——各司其职，最终统一合成 **P3-技术方案.md**。

P4 · 方案审查

验证架构合理性、技术选型依据、数据模型覆盖度、风险缓解策略。

P5 · 编码计划

将方案拆解为 2-5 分钟可完成的原子任务，组织为有向无环图。同层任务可并行，跨层串行。每个任务生成独立的 Story 文件 (**stories/0XX-*.md**)，只携带相关上下文片段。

P6 · 计划审查

检查任务粒度、依赖完整性、Story 自包含性。

P7 · 测试用例

按等价类划分和边界值分析，生成三类用例：功能（50-60%）、边界（20-30%）、异常（15-20%）。前端项目额外生成 CDP 截图测试。

P8 · 用例审查

审查覆盖完整性与场景合理性。

P9 · 编码执行

按拓扑层级顺序执行 Story 文件，严格遵循 RED→GREEN→REFACTOR→COMMIT 循环。每批完成后自动触发代码审查。

P10 · 代码审查（双重硬门禁）

阶段	内容	规则
Stage 1	规约合规性	代码是否实现了计划中的每一项？缺失 → 不通过
Stage 2	代码质量	风格、性能、安全性、可维护性、测试覆盖

先保证**做对的事**，再保证**把事做对**。Stage 1 不通过则不能进入 Stage 2。

P11 · 测试执行

全量执行 P7 中的测试用例，失败用例自动创建缺陷追踪条目，关联到对应 Story 和 commit。产出 **P11-测试报告.md**。

P12 · 测试审查

评估通过率、缺陷趋势、高风险区域，决定是否需要修复后重新测试。

P13 · Wiki 文档

三阶段流程（大纲设计 → 内容写作 → 一致性审查）自动生成项目 Wiki：架构概览、API 参考、部署指南。

P14 · Wiki 审查

核查代码引用准确性、术语一致性、文档完整性。

P15 · 成果提交

创建 PR → 清理 Worktree → 归档迭代记录 → 目录重命名。交付即完结。

架构

借鉴责任链和策略模式，编排器与阶段技能严格分离：

组件	职责	边界
ideal-flow-control （编排器）	读取 flow state → 判断当前阶段 → 调用阶段 Skill → 更新 flow state	绝不直接操作文件、读代码、写文档、做技术决策
阶段 Skill （执行者）	接收上下文 → 启动子智能体 → 产出交付物 → 返回摘要	不感知整体流程，只负责当前阶段

所有实质工作通过子智能体完成，每个环节的输出质量由独立上下文保证。

量化效果

以**用户认证模块**（邮箱注册/登录 + Google/GitHub OAuth + 密码重置 + JWT + 角色权限）为例：

指标	传统方式	ideal-dev-workflow	提升
需求耗时	1.5 天	0.3h	83% ↓
方案耗时	4h	0.5h	87% ↓
编码耗时	6h	2.5h	58% ↓
测试耗时	3h	1.0h	67% ↓
总耗时（含修复）	~12h	~4.6h	62% ↓
沟通轮次	5 轮+	0 轮	100% ↓
需求偏差率	~40%	<5%	88% ↓
缺陷逃逸率	~25%	<8%	68% ↓
测试覆盖率	~30%	>85%	2.8×
文档完整度	低	100%	—
决策可追溯	0%	100%（13 份过程文档）	—

同一需求，同一开发者，同一技术栈（Go 1.22 + PostgreSQL + Redis + JWT）。唯一变量：是否使用 ideal-dev-workflow。

适用场景

角色	典型场景	核心收益
后端开发	接到新 API 需求	P1 锁定需求，偏差率 <5%
前端开发	复杂交互组件开发	P7 + P11 系统化测试，回归率降 70%
全栈工程师	独立负责完整功能	Story 分片 + 拓扑执行，上下文消耗降 60%
Tech Lead	方案评审与技术决策	P3 6 子智能体并行调研，方案完整性提升
QA 工程师	手工测试效率低	三类系统化用例 + 自动执行，缺陷逃逸率 <8%
文档工程师	项目文档编写	三阶段 Wiki + 一致性审查，文档即代码

快速开始

前置要求

- Node.js ≥ 18
- Git ≥ 2.30（支持 Worktree）
- Claude Code（支持 Plugin 架构）

安装

```
# 添加 Marketplace
claude plugin marketplace add https://github.com/MTleen/ideal-dev-workflow

# 安装插件
claude plugin install ideal-dev-workflow@ideal-dev-workflow
```

启动第一个迭代

```
cd your-project
```

在 Claude Code 中输入：

```
/ideal-init
```

然后用自然语言描述你的需求：

```
我要做一个用户认证系统，支持邮箱注册登录、OAuth 登录、密码重置和角色权限管理。
```

编排器会自动启动 P1 苏格拉底对话，之后的工作流自动推进。完整的流程状态随时在 [流程状态.md](#) 中可见。

技能速查表

Skill	阶段	用途
ideal-init	—	项目初始化与配置生成
ideal-requirement	P1	苏格拉底式需求对话
ideal-dev-solution	P3	6 子智能体并行方案设计
ideal-dev-plan	P5	原子任务拆分 + Mermaid DAG
ideal-test-case	P7	功能/边界/异常三类测试用例
ideal-dev-exec	P9	TDD 循环编码执行
ideal-code-review	P10	规约合规性 → 代码质量双重审查
ideal-test-exec	P11	全量测试执行 + 缺陷追踪
ideal-wiki	P13	三阶段 Wiki 文档生成
ideal-delivery	P15	PR 创建 + Worktree 清理 + 归档

Skill	阶段	用途
ideal-flow-control	—	阶段流转状态管理
ideal-yolo	—	多视角自动评审 + 熔断器
ideal-deep-research	—	企业级深度调研
ideal-debugging	—	系统调试与根因分析

目录结构

```
ideal-dev-workflow/
├── .claude-plugin/marketplace.json  # Marketplace 索引
├── plugins/ideal-dev-workflow/      # 插件主体
│   ├── .claude-plugin/plugin.json  # 插件元数据
│   └── skills/                     # 14 个 Skill
│       ├── ideal-init/             # 项目初始化
│       ├── ideal-requirement/      # P1 需求分析
│       ├── ideal-dev-solution/     # P3 技术方案
│       ├── ideal-dev-plan/         # P5 编码计划
│       ├── ideal-test-case/        # P7 测试用例
│       ├── ideal-dev-exec/         # P9 编码执行
│       ├── ideal-code-review/      # P10 代码审查
│       ├── ideal-test-exec/        # P11 测试执行
│       ├── ideal-wiki/             # P13 Wiki 文档
│       ├── ideal-delivery/         # P15 成果提交
│       ├── ideal-flow-control/     # 阶段流转编排
│       ├── ideal-yolo/             # 自动审查模式
│       ├── ideal-deep-research/    # 深度调研
│       └── ideal-debugging/        # 系统调试
├── package.json
├── CHANGELOG.md
├── README.md
└── LICENSE
```

理念

ideal-dev-workflow 的核心信念：

AI 可以写出能跑的代码，但写出可靠的代码需要工程纪律。

15 个阶段不是多出来的流程，而是把隐性风险变成显性检查的安全网。每一阶段都在回答同一个问题：**我们怎么知道这一步做对了？**

- TDD 保证代码做对了
- 苏格拉底对话保证做对的事
- 对抗式审查保证独立验证
- 上下文隔离保证决策在充分信息下做出

- 全链路文档保证一切可追溯
-

配套资源

- **GitHub 仓库**: github.com/MTleen/ideal-dev-workflow
 - **演示视频**: 10 分钟完整讲解 (横屏 1920×1080, 含配音和字幕)
 - **演讲 PPT**: 30 页幻灯片 ([slides/ideal-dev-workflow/](#))
 - **讲稿**: 逐页对应的 10 分钟演讲文本
 - **提示词模板**: 6 个阶段的即用 prompt
-

License

MIT